

关系数据库标准语言

SQL语言特点

- 综合统一
- 高度非过程化
- 面向集合的操作方式
- 以同一种语法结构提供两种使用方式
 - 自含式语言
 - 嵌入式语言
- 语言简捷，易学易用

数据定义

- 定义表：**CREATE TABLE** <表名> (<列名> <数据类型> [<列级完整性约束条件>] [, <列名> <数据类型> [<列级完整性约束条件>]] ... [, <表级完整性约束条件>]);
- 修改表：**ALTER TABLE** <表名> [ADD <新列名> <数据类型> [完整性约束]] [DROP <列名>] [DROP <完整性约束名>] [MODIFY <列名> <数据类型>]
- 删除表：**DROP TABLE** <表名>

数据更新

数据插入INSERT:

INSERT INTO <表名> [(<属性列1> [, <属性列2>] ...)] **VALUES** (<常量1> [, <常量2>] ...);

eg.

- Insert into Students Values('200011', '张三', 19);
- Insert into Students(sid, name) Values('200013', '王五');

数据修改UPDATE:

UPDATE <表名> **SET** <列名>=<表达式> [, <列名>=<表达式>] ... [**WHERE** <条件>];

eg.

- 根据A属性改B型: Update Students Set age = 18 where sid = '200011'
- 单属性自己遍型:
 - Update Students Set age = age + 1;
 - Update Students Set sid = '200012' where sid = '200011';

数据删除DELETE:

与表的删除区分!!! DELETE和DROP!!!

DELETE FROM <表名> [**WHERE** <条件>];

注意：Delete from Students删除了Students 的所有元组，但保留了表 的结构、属性和索引

数据查询

数据选择SELECT:

SELECT A1, A2, ..., An #3: what to return

FROM R1, R2, ..., Rn #1: relations to query

WHERE condition #2: combine, filter relations

- SELECT * FROM ... = Select all attributes
- 用关系代数表达： $\Pi_{A1, A2, \dots, An}(\sigma_{condition}(R1 \times R2 \times \dots \times Rn))$
- The result of a SELECT query is also a relation.
- 用C语言（伪代码）表示：

```
Result = {}
for x1 in R1 do
  for x2 in R2 do
    ....
    for xn in Rn do
      if conditions(x1,..., xn)
        then Result = Result union {(x1.a1, x1.a2, ..., xn.an)}
return Result
```

C

DISTINCT:

- **DISTINCT** 用于 去除查询结果中的重复行（**duplicates**）。
- **SELECT DISTINCT city FROM Station;**
- 与**COUNT**使用： **SELECT COUNT(DISTINCT city) FROM Station;**：统计城市的种类数量，而不是所有记录的总数。
- 如果只想对部分列去重（比如某一列）
 - SQL 标准并不支持“单列 distinct”，需要配合 **GROUP BY**：
 - **SELECT city FROM Station GROUP BY city;** 等价于 **SELECT DISTINCT city FROM Station;**

SUBQUERY子查询操作符 (IN/EXISTS/ALL/ANY):

- **IN** - 判断某个值 是否在一个集合（或子查询的结果）中。
 - eg: 查找选修了 任意一个名称中包含“GIS”的选修课程 的学生姓名：

```

SELECT name --投影到学生姓名
FROM Student --选择Student表
WHERE sid IN ( --这个学生的sid属于以下筛选结果中
    SELECT sid --选择出选课记录中有cid符合以下筛选结果的
    FROM SC
    WHERE cid IN (
        SELECT cid FROM Course WHERE cname LIKE '%GIS%' --选择出名称中包含'GIS'的课
    )
);

```

- **EXISTS** - 判断子查询 是否返回至少一条记录。通常用于存在性判断。
 - eg: 查找 选修过至少一门课程 的学生:

```

SELECT name
FROM Student S
WHERE EXISTS (
    SELECT 1 --1为形式，实际内容与逻辑无关
    FROM SC
    WHERE SC.sid = S.sid --学生表中的名字和选课记录中的名字匹配
);

```

- **ALL** - 用于 比较运算符 + **ALL** 的形式，表示 与子查询结果的所有值都比较成立。
 - eg: 查找成绩比课程 'GEOS3028M' 所有学生都高的学生（就是成绩最高的学生）

```

SELECT sid
FROM SC
WHERE grade >= ALL (
    SELECT grade FROM SC WHERE cid = 'GEOS3028M' -- 筛选出所有成绩
);

```

- **ANY** - 与 **ALL** 相反，表示 与子查询结果中至少一个值比较成立。
 - eg: 查找成绩高于课程 **GEOS3028M** 中任意一个学生的学生:

```

SELECT sid
FROM SC
WHERE grade > ANY (
    SELECT grade FROM SC WHERE cid = 'GEOS3028M'
);

```

集合操作符 (UNION/INTERSECT/EXCEPT)

作用说明表

运算符	含义 (集合操作)	SQL 语法作用	是否去重 (默认)
UNION	并集	合并两个结果集的所有不同记录	去重
INTERSECT	交集	返回两个结果集中共有的记录	去重
EXCEPT	差集	返回第一个结果集中不在第二个中的记录	去重
UNION ALL	并集不去重	合并结果集 (不去重)	不去重

语法格式:

SQL

```
SELECT column_list --视为第一个集合
FROM table_A
[WHERE 条件]
UNION / INTERSECT / EXCEPT --集合操作符
SELECT column_list --视为第二个集合
FROM table_B
[WHERE 条件];
```

注意事项 (条件) :

- 1. 两个查询必须有相同的列数;
 - 2. 对应列的数据类型必须兼容;
 - 3. 列名由第一个 **SELECT** 决定。
- LIMIT** - 返回结果数

SQL 聚合函数与极值查询

分类	函数	功能
聚合函数 (Aggregate Functions)	MAX()	返回最大值
	MIN()	返回最小值
	AVG()	返回平均值
	SUM()	返回总和
	COUNT()	返回计数 (行数)

注意事项:

- 1. 输入是多行, 输出是单个值;
- 2. 只能在 **SELECT** 子句中使用;
- 3. 常与 **GROUP BY** 一起用;

4. 聚合函数忽略 **NULL** 值。

ORDER BY - **ORDER BY GPA DESC** 按__排序

- **DESC**: 从高到低排序 (降序)
- **ASC**: 从低到高排序 (升序)

连接符 (JOIN Operators)

Inner Join (内连接)

只返回两个表中匹配成功的行。(默认的JOIN)

SQL

```
-- 1. 旧写法 (等价连接)
SELECT R.A, S.B
FROM R, S
WHERE R.A = S.A;

-- 2. 标准写法 (推荐)
SELECT R.A, S.B
FROM R INNER JOIN S
ON R.A = S.A;

-- 3. USING 写法 (当列名相同)
SELECT R.A, S.B
FROM R INNER JOIN S
USING (A);
```

Natural Join (自然连接)

自动按两个表中同名的列匹配并连接。

SQL

```
SELECT R.A, S.B
FROM R NATURAL JOIN S;
```

Outer Join (外连接)

与 Inner Join 不同, 它会保留未匹配的行, 并用 **NULL** 填充缺失部分。

类型	含义	举例
LEFT OUTER JOIN	保留左表全部行	即使右表无匹配, 也保留左表行, 右侧填 NULL
RIGHT OUTER JOIN	保留右表全部行	即使左表无匹配, 也保留右表行, 左侧填 NULL
FULL OUTER JOIN	保留左右表全部行	任一表无匹配的行都会出现, 用 NULL 补齐

SQL

```
-- eg. 左外连接
SELECT R.A, S.B
FROM R LEFT OUTER JOIN S
ON R.A = S.A;
```

USING VS ON

写法	特点
ON R.a = S.a	可以连接不同列名
USING (a)	两个表的列名必须相同，自动只保留一个该列

Aggregation 聚合查询

整体语法结构：

```
SELECT A1, A2, ..., An
FROM R1, R2, ..., Rn
WHERE C1
GROUP BY Ai, Aj, ..., Ak
HAVING C2;
```

SQL

子句作用与执行顺序：

步骤	子句	作用	说明
1	FROM / WHERE	选择参与的表并筛选符合条件的行	对单行起作用
2	GROUP BY	按一个或多个属性分组	每组是一类
3	HAVING	对每一组进行筛选	对分组结果起作用
4	SELECT	输出每组的属性和聚合结果	包含分组列 + 聚合函数
5	ORDER BY (可选)	对输出排序	最后执行

HAVING VS WHERE :

子句	作用阶段	作用对象	示例
WHERE	分组前	行 (单条记录)	WHERE price > 10
HAVING	分组后	组 (聚合结果)	HAVING SUM(price * quantity) > 100

⚠ 不要单独使用 **HAVING** 而没有 **GROUP BY**!

⚠ 聚合函数只能在**HAVING**中使用!

SELECT 子句限制 (很重要)

在有 GROUP BY 的查询中：

SELECT 中的字段只能是

- GROUP BY 中的分组字段

- 或者聚合函数结果

✗ 错误示例:

```
SELECT product, price FROM Purchase GROUP BY product;
```

SQL

✓ 正确示例:

```
SELECT product, AVG(price) FROM Purchase GROUP BY product;
```

SQL

NULL VALUES

NULL:

- **NULL** 表示“未知”(unknown) 或 “不存在”(undefined) 的值。
- 它 不是 0、不是空字符串 ""、也不是 False。

NULL 在表达式中的表现:

任何包含 **NULL** 的运算结果都是 **NULL**。

布尔逻辑中的三值逻辑 (Three-Valued Logic)

For boolean operations, in SQL there are three values

- FALSE = 0
- UNKNOWN = 0.5
- TRUE = 1

示例:

WHERE x = 'Joe'

- 如果 **x** 为 'Joe' → TRUE
- 如果 **x** 为 'Bob' → FALSE
- 如果 **x** 为 **NULL** → UNKNOWN

三值逻辑运算规则:

运算	规则
C1 AND C2	= min(C1, C2)
C1 OR C2	= max(C1, C2)
NOT C1	= 1 - C1

- 在 **SQL** 查询中的影响: 在 **WHERE** 子句中, 只有返回 **TRUE** 的行才会被选中。所以 **NULL** 值不会被 **WHERE** 条件选中, 除非显式地处理。

检测 NULL 的正确方法:

```
x IS NULL  
x IS NOT NULL
```

SQL

示例:

```
SELECT * FROM Student  
WHERE GPA >= 3.5 OR GPA < 3.5; --无法选中GPA为NULL的学生
```

SQL

VS

```
SELECT * FROM Student  
WHERE GPA >= 3.5 OR GPA < 3.5 OR GPA IS NULL; --可以选中GPA为NULL的学生
```

SQL